

1 Iris data prediction using K-Nearest Neighbour Classification

```
[1]: # Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
[2]: # Load the Iris dataset
# dataset = pd.read_csv("iris.csv")

# load iris data and store in dataframe

from sklearn import datasets

iris = datasets.load_iris()

dataset = pd.DataFrame(data=iris.data, columns=iris.feature_names)
dataset["target"] = iris.target
dataset
```

```
[2]:      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1             3.5             1.4             0.2
1                4.9             3.0             1.4             0.2
2                4.7             3.2             1.3             0.2
3                4.6             3.1             1.5             0.2
4                5.0             3.6             1.4             0.2
..                ...             ...             ...             ...
145              6.7             3.0             5.2             2.3
146              6.3             2.5             5.0             1.9
147              6.5             3.0             5.2             2.0
148              6.2             3.4             5.4             2.3
149              5.9             3.0             5.1             1.8
```

```
      target
0          0
1          0
2          0
3          0
4          0
..         ...
145        2
146        2
147        2
148        2
149        2
```

[150 rows x 5 columns]

```
[3]: # Extract the feature data
X = dataset.iloc[:, :-1].values
X[:10]
```

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 7th September 2026

```
[3]: array([[5.1, 3.5, 1.4, 0.2],
          [4.9, 3. , 1.4, 0.2],
          [4.7, 3.2, 1.3, 0.2],
          [4.6, 3.1, 1.5, 0.2],
          [5. , 3.6, 1.4, 0.2],
          [5.4, 3.9, 1.7, 0.4],
          [4.6, 3.4, 1.4, 0.3],
          [5. , 3.4, 1.5, 0.2],
          [4.4, 2.9, 1.4, 0.2],
          [4.9, 3.1, 1.5, 0.1]])
```

```
[4]: # Extract the target labels
y = dataset.iloc[:, -1].values
y[:10]
```

```
[4]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
[5]: # Split the data into training and test sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
[6]: # Preview the training features
X_train[:10]
```

```
[6]: array([[6.5, 3. , 5.8, 2.2],
          [4.9, 3.6, 1.4, 0.1],
          [6.5, 3. , 5.2, 2. ],
          [5.3, 3.7, 1.5, 0.2],
          [5.1, 3.8, 1.6, 0.2],
          [5.7, 2.5, 5. , 2. ],
          [5.4, 3. , 4.5, 1.5],
          [5.7, 3. , 4.2, 1.2],
          [5.9, 3. , 4.2, 1.5],
          [6.7, 2.5, 5.8, 1.8]])
```

```
[7]: # Preview the test features
X_test[:10]
```

```
[7]: array([[6.5, 3. , 5.5, 1.8],
          [5.6, 2.8, 4.9, 2. ],
          [6. , 2.7, 5.1, 1.6],
          [6.3, 3.3, 4.7, 1.6],
          [6.1, 3. , 4.9, 1.8],
          [5. , 3.2, 1.2, 0.2],
          [5.8, 4. , 1.2, 0.2],
          [6.7, 3.1, 4.7, 1.5],
          [5.5, 2.4, 3.7, 1. ],
          [4.4, 3. , 1.3, 0.2]])
```

```
[8]: # Preview the training labels
y_train[:10]
```

```
[8]: array([2, 0, 2, 0, 0, 2, 1, 1, 1, 2])
```

```
[9]: # Preview the test labels
y_test[:10]
```

```
[9]: array([2, 2, 1, 1, 2, 0, 0, 1, 1, 0])
```

```
[10]: # Scale the training features
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_train[:10]
```

```
[10]: array([[ 0.85086782, -0.13529958,  1.2011581 ,  1.35065636],
 [-1.08246498,  1.27652212, -1.30066233, -1.41204983],
 [ 0.85086782, -0.13529958,  0.86000077,  1.08754149],
 [-0.59913178,  1.51182574, -1.24380277, -1.2804924 ],
 [-0.84079838,  1.74712935, -1.18694322, -1.2804924 ],
 [-0.11579858, -1.31181766,  0.74628166,  1.08754149],
 [-0.47829848, -0.13529958,  0.46198389,  0.4297543 ],
 [-0.11579858, -0.13529958,  0.29140522,  0.03508198],
 [ 0.12586802, -0.13529958,  0.29140522,  0.4297543 ],
 [ 1.09253442, -1.31181766,  1.2011581 ,  0.82442661]])
```

```
[11]: # Scale the test features
X_test = scaler.transform(X_test)
X_test[:10]
```

```
[11]: array([[ 0.85086782, -0.13529958,  1.03057944,  0.82442661],
 [-0.23663188, -0.60590681,  0.68942211,  1.08754149],
 [ 0.24670132, -0.84121043,  0.80314122,  0.56131174],
 [ 0.60920122,  0.57061127,  0.575703 ,  0.56131174],
 [ 0.36753462, -0.13529958,  0.68942211,  0.82442661],
 [-0.96163168,  0.33530765, -1.41438144, -1.2804924 ],
 [ 0.00503472,  2.21773659, -1.41438144, -1.2804924 ],
 [ 1.09253442,  0.10000404,  0.575703 ,  0.4297543 ],
 [-0.35746518, -1.54712128,  0.00710744, -0.22803289],
 [-1.68663148, -0.13529958, -1.35752188, -1.2804924 ]])
```

```
[12]: # Train the KNN classifier
from sklearn.neighbors import KNeighborsClassifier

classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(X_train, y_train)
```

```
[12]: KNeighborsClassifier()
```

```
[13]: # Predict the test labels
y_pred = classifier.predict(X_test)
y_pred
```

```
[13]: array([2, 2, 2, 1, 2, 0, 0, 1, 1, 0, 2, 1, 2, 0, 2, 2, 1, 0, 2, 2, 1, 0,
          2, 0, 2, 1, 2, 0, 2, 1])
```

```
[14]: # Compute the confusion matrix
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)
```

```
[14]: array([[ 8,  0,  0],
          [ 0,  8,  1],
          [ 0,  0, 13]])
```

```
[15]: # Print the classification report
from sklearn.metrics import classification_report

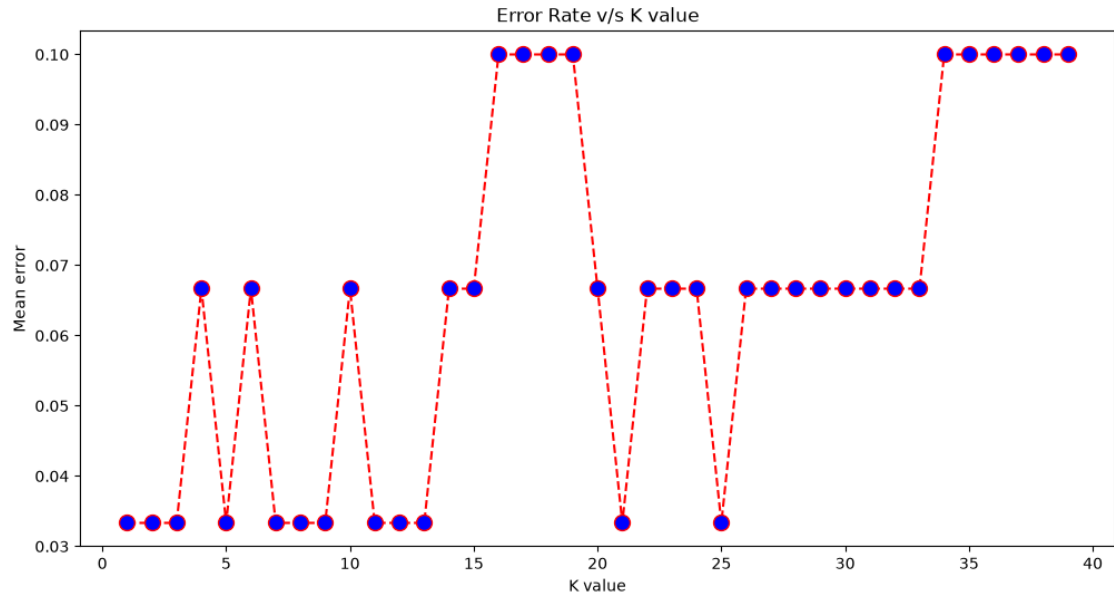
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	1.00	0.89	0.94	9
2	0.93	1.00	0.96	13
accuracy			0.97	30
macro avg	0.98	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

```
[16]: # Test different K values and collect errors
errors = []

for i in range(1, 40):
    classifier = KNeighborsClassifier(n_neighbors=i)
    classifier.fit(X_train, y_train)
    y_pred = classifier.predict(X_test)
    errors.append(np.mean(y_test != y_pred))
```

```
[17]: # Plot error rate against K value
plt.figure(figsize=(12, 6))
plt.plot(range(1, 40), errors, color="red", linestyle="dashed", marker="o",
        markerfacecolor="blue", markersize=10)
plt.title("Error Rate v/s K value")
plt.xlabel("K value")
plt.ylabel("Mean error")
plt.show()
```



Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date: 7th September 2026